DESIGN DOCUMENTATION

Final Project

Legends Of Valor

Object-Oriented Software Principles and Design

Group Members:

Chris Mary Benson - U56085268

Serena Noboudem - U63767551

Ziyang Wang - U12285471

CONTENTS

1. INTRODUCTION

1.1 Purpose

This document presents the architectural and object-oriented design of a game system that supports two games (Monsters and Heroes, Legends of Valor) operating on a shared infrastructure. It details the system's structure, core components, and key design decisions that enable code reuse, maintainability, and scalability, while ensuring consistent behavior across both games, respecting the rules of each game.

2. SYSTEM OVERVIEW

2.1 Current Framework

The framework is designed to provide a reusable and extensible foundation for the creation of games similar to Monsters and Heroes. It leverages object-oriented principles to promote modularity, maintainability, and scalability.

2.1.1 How Design Goals were Achieved

- **Implementation of Design Patterns:**

  - ➢ **Factory Pattern -**
    I. LovTile uses factory methods (heroNexus, monsterNexus, terrain) to centralize tile creation

  - ➢ **Strategy Pattern:**

    I. The Strategy Pattern is used in our infrastructure to model **different combat behaviors** of the heroes and monsters, such as physical attacks and spell casting, in a flexible and extensible manner. Instead of hard-coding combat logic within hero or monster classes, we have changed it such that the combat behavior is encapsulated into separate strategy objects. A common combat interface (i.e. FightStrategy) defines the contract for all combat behaviors.

The concrete implementations such as AttackStrategy and SpellCastingStrategy provide specific logic for different types of combat. At runtime, a hero or monster is assigned a particular fight strategy, enabling dynamic selection.

II. TerrainGenerator defines a strategy interface so LovBoard can swap randomization policies without changing board code.

➢ **Observer Pattern:**

I. The Observer Pattern is used to efficiently manage the distribution of rewards to all heroes in a party whenever one hero defeats a monster. In this design, the heroes act as observers (or subscribers), while the PartyController serves as the publisher (or subject). This approach decouples the reward distribution logic from the hero classes and centralizes it within the party controller.

The PartyController class maintains a list of registered heroes and provides methods to add or remove observers. When a hero defeats a monster, the PartyController is notified and triggers an update event. This event invokes a method on each subscribed hero, granting them the appropriate rewards.

➢ **Proxy Pattern -**

I. The Proxy Pattern is used here to manage battle interactions between heroes and monsters by acting as an intermediary that controls access to the underlying Battle object. In this design, the proxy handles preconditions for attacks, such as verifying whether a hero has the required weapons or spells and checking that both the hero and the monster are within valid attack range. The Battle object itself is initialized and encapsulated within the proxy, ensuring that all combat logic passes through a controlled interface.

By using a proxy, the system centralizes validation and access control, allowing the actual Battle class to focus solely on executing attacks and resolving combat outcomes.

Heroes and monsters do not need to handle preconditions individually; the proxy abstracts these checks and ensures that only valid attack actions reach the Battle object.

- ➢ **State Pattern -**
  I. Used to encapsulate the game's flow control (HeroTurnState, MonsterTurnState, RoundEndState).
  II. Replaced monolithic loops with discrete state classes, ensuring strict enforcement of turn order (Heroes → Monsters → Regen), and allowing for easy addition of new game phases without modifying the main loop.
- ➢ **Command Pattern -**
  I. Decouples user input from game logic. Inputs are parsed into Command objects (MoveCommand, TeleportCommand, AttackCommand, RecallCommand).
  II. Encapsulates complex validation rules (like the "Pass Behind" blocking rule) inside the command itself rather than the controller, and allows for easy extension of player capabilities.

- ➢ **Singleton Pattern -**
  I. Used for the GameDatabase to ensure a single, global access point for loading and retrieving static game assets (Monsters, Items, Spells) across both game modes, preventing redundant file I/O and memory usage.

- **Scalability & Reusability**

  - ➢ LovBoard injects **Random** and **TerrainGenerator**, allowing different map sizes, spawn rules, and terrain policies without changing board logic.

  - ➢ TerrainGenerator defines a strategy interface; new terrain algorithms only need to implement generate to be reused.

  - ➢ RandomTerrainGenerator centralizes weights and required tile placement, so rule changes only require updating probabilities.

- ➢ LovTile uses factory methods for nexus and terrain creation, ensuring consistency and reuse.

- ➢ MonsterAI isolates movement and decision logic, making it easy to replace or extend AI behaviors.

- ➢ The MarketController and InventoryController from Monsters and Heroes was constructed in such a way that it was easily reusable in Legends of Valor.

- ➢ The fight strategies are reusable in two ways: they can easily be called by any games structure and they can also be used for any type of living entities.

- ➢ The fight strategies are also extensible, if any game requires more fight strategies, we can simply add more fight action classes that implement the same interface.

2.3 Changes from Previous Framework

- **Transition to State-Driven Architecture:** The previous framework relied on linear procedural loops to manage game turns. In Phase 3, we refactored the Legends of Valor engine to use the State Design Pattern. The LovGameController now delegates the execution flow to specific state classes (HeroTurnState, MonsterTurnState). This change rigidly enforces the game rules, specifically that all heroes must act before any monster can move, and separates the "Spawning/Regen" logic into its own RoundEndState.
- **Decoupled Input Handling (Command Pattern):** We moved away from executing game logic directly inside the input parsing block. We implemented the Command Design Pattern to treat user actions as objects (LovCommand). This allowed us to implement complex validation rules, such as the "Monster Block" rule, which prevents a hero from moving to a row behind a monster in their lane, directly inside the MoveCommand,without cluttering the main game loop.
- **Modular Integration via Proxies:** To integrate the work of different team members (Board Logic and Combat Logic), we introduced the LoVBattleProxy. This class acts as an adapter that bridges the new grid-based positioning system with the existing turn-based battle engine. It allows the MOBA game mode to reuse the robust combat formulas and inventory systems from "Monsters and Heroes" without code duplication.

3. ARCHITECTURAL DESIGN

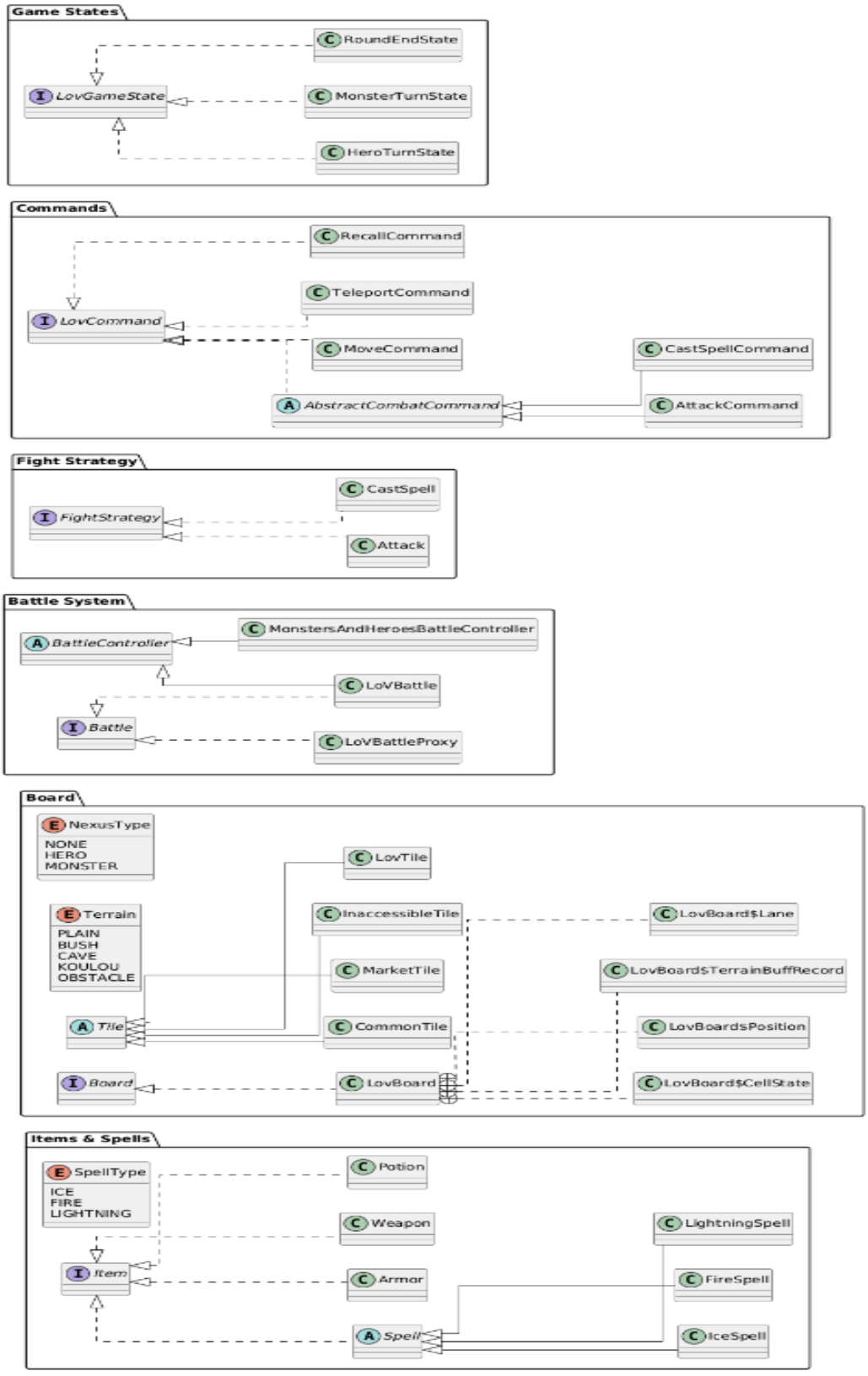
3.1 Components

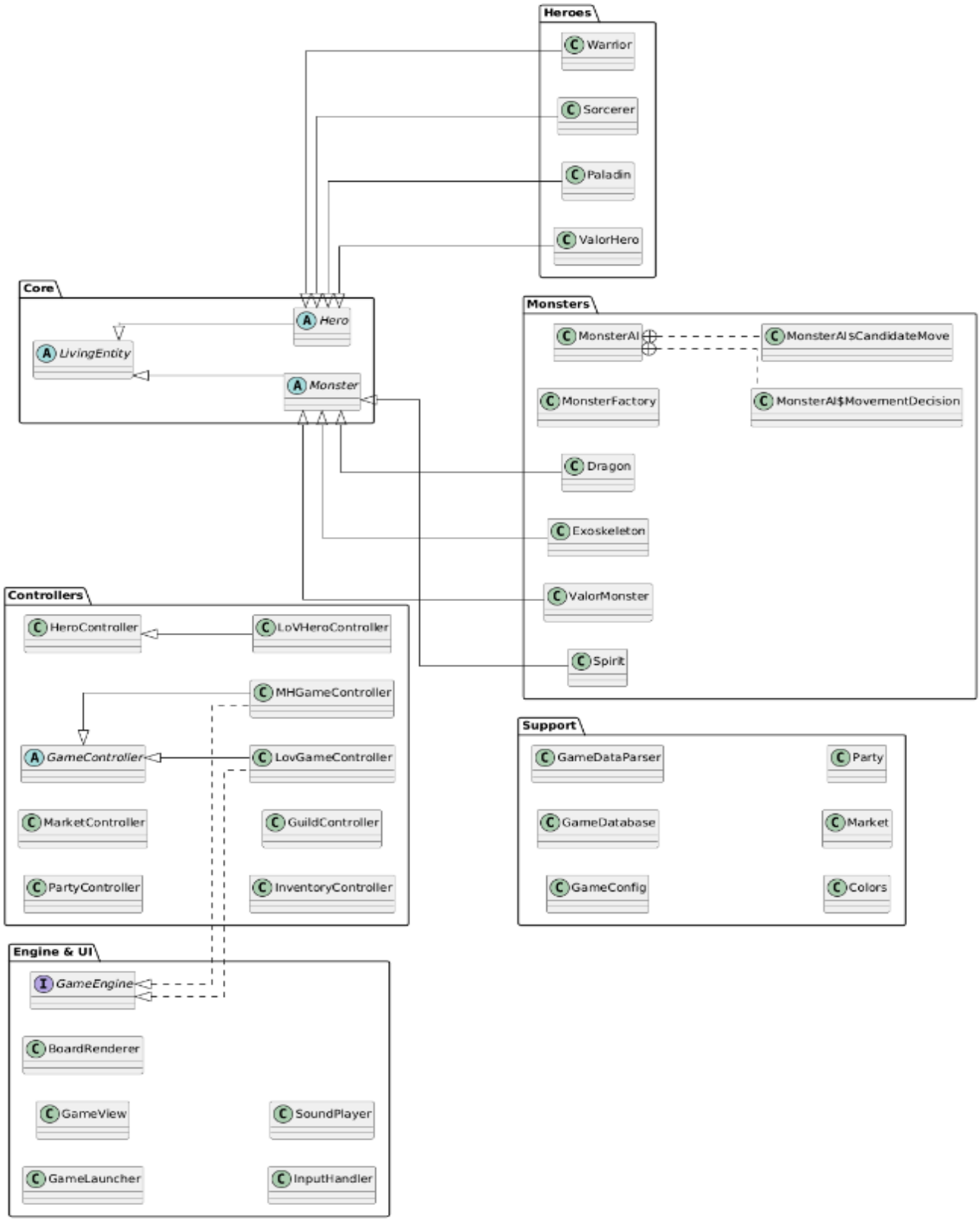
```
LegendsOfValors
└── LegendsOfValors
    ├── data_files/
    │   ├── Armory.txt
    │   ├── Dragons.txt
    │   ├── Exoskeletons.txt
    │   ├── FireSpells.txt
    │   ├── IceSpells.txt
    │   ├── Initial.txt
    │   ├── LightningSpells.txt
    │   ├── Paladins.txt
    │   ├── Potions.txt
    │   ├── Sorcerers.txt
    │   ├── Spirits.txt
    │   ├── Warriors.txt
    │   └── Weaponry.txt
    ├── soundfiles/
    │   └── themes.wav
    └── src/
        ├── GameLauncher.java
        ├── Main.java
        ├── core/
        │   ├── interfaces/
        │   │   ├── Board.java
        │   │   ├── FightStrategy.java
        │   │   ├── GameEngine.java
        │   │   ├── HeroEventNotifier.java
        │   │   └── HeroObserver.java
        │   ├── model/
        │   │   ├── GameDatabase.java
        │   │   ├── Party.java
        │   │   ├── entity/
        │   │   │   ├── Dragon.java
        │   │   │   ├── Exoskeleton.java
        │   │   │   ├── Hero.java
        │   │   │   ├── LivingEntity.java
        │   │   │   ├── Monster.java
        │   │   │   ├── Paladin.java
        │   │   │   ├── Sorcerer.java
```

```
│   │   │   ├── Spirit.java
│   │   │   ├── Warrior.java
│   │   │   └── decorator/
│   │   │       └── HeroDecorator.java
│   │   ├── item/
│   │   │   ├── Armor.java
│   │   │   ├── Item.java
│   │   │   ├── Potion.java
│   │   │   ├── Weapon.java
│   │   │   └── spell/
│   │   │       ├── FireSpell.java
│   │   │       ├── IceSpell.java
│   │   │       ├── LightningSpell.java
│   │   │       └── Spell.java
│   │   ├── market/
│   │   │   └── Market.java
│   │   └── world/
│   │       ├── Board.java
│   │       ├── CommonTile.java
│   │       ├── InaccessibleTile.java
│   │       ├── MarketTile.java
│   │       └── Tile.java
│   └── util/
│       ├── Colors.java
│       ├── GameConfig.java
│       ├── GameDataParser.java
│       └── SoundPlayer.java
└── games/
    ├── commoncontrollers/
    │   ├── BattleController.java
    │   ├── GameController.java
    │   ├── HeroController.java
    │   ├── InputHandler.java
    │   ├── InventoryController.java
    │   ├── MarketController.java
    │   ├── PartyController.java
    │   └── actions/
    │       ├── Attack.java
    │       └── CastSpell.java
    ├── legendsofvalors/
    │   ├── commands/
    │   │   ├── AbstractCombatCommand.java
    │   │   ├── AttackCommand.java
    │   │   ├── CastSpellCommand.java
```

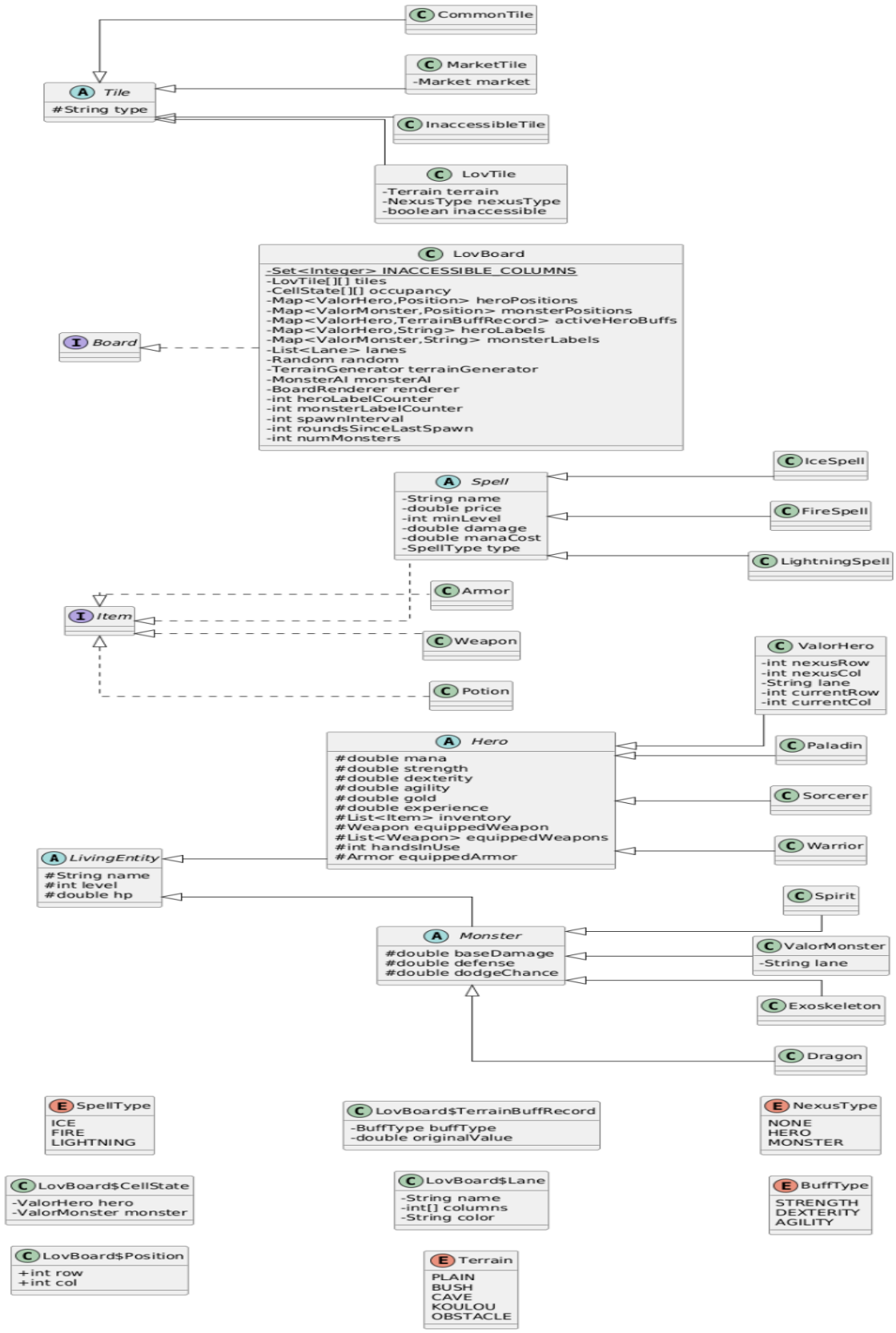
```
│   │   ├── MoveCommand.java
│   │   ├── RecallCommand.java
│   │   └── TeleportCommand.java
│   ├── controller/
│   │   ├── LovGameController.java
│   │   ├── LoVHeroController.java
│   │   └── battle/
│   │       ├── LoVBattle.java
│   │       └── LoVBattleProxy.java
│   ├── interfaces/
│   │   ├── Battle.java
│   │   ├── LovCommand.java
│   │   └── LovGameState.java
│   ├── model/
│   │   ├── ValorHero.java
│   │   ├── ValorMonster.java
│   │   ├── factory/
│   │   │   └── MonsterFactory.java
│   │   └── world/
│   │       ├── BoardRenderer.java
│   │       ├── LovBoard.java
│   │       ├── LovTile.java
│   │       ├── MonsterAI.java
│   │       └── generator/
│   │           ├── RandomTerrainGenerator.java
│   │           └── TerrainGenerator.java
│   ├── states/
│   │   ├── HeroTurnState.java
│   │   ├── MonsterTurnState.java
│   │   └── RoundEndState.java
│   ├── testing/
│   │   └── LovGameSmokeTest.java
│   └── util/
│       └── GameConfig.java
└── monstersandheroes/
    ├── controller/
    │   ├── GuildController.java
    │   ├── MHGameController.java
    │   └── MonstersAndHeroesBattleController.java
    ├── model/
    │   └── Board.java
    └── view/
        ├── Colors.java
        └── GameView.java
```

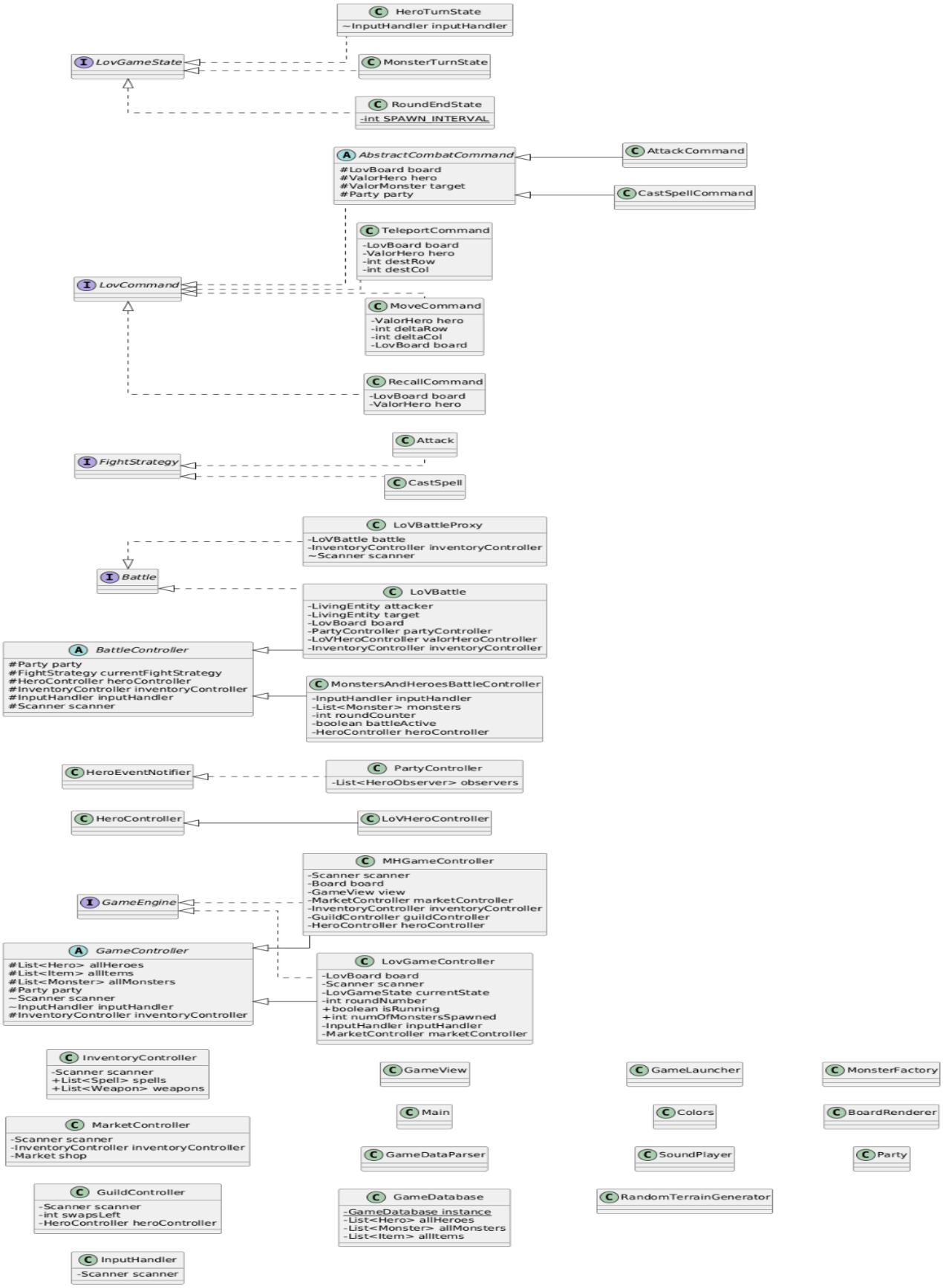
3.2 Class Hierarchy with Packages




Heroes
Warrior
Sorcerer
Paladin
ValorHero
Core
Hero
LivingEntity
Monster
Monsters
MonsterAI
MonsterAIsCandidateMove
MonsterFactory
MonsterAI$MovementDecision
Dragon
Exoskeleton
ValorMonster
Spirit
Controllers
HeroController
LoVHeroController
MHGameController
GameController
LoVGameController
MarketController
GuildController
PartyController
InventoryController
Support
GameDataParser
Party
GameDatabase
Market
GameConfig
Colors
Engine & UI
GameEngine
BoardRenderer
GameView
SoundPlayer
GameLauncher
InputHandler

3.3 UML Diagram




HeroTurnState
~InputHandler inputHandler
LovGameState
MonsterTurnState
RoundEndState
-int SPAWN_INTERVAL
AbstractCombatCommand
#LovBoard board
#ValorHero hero
#ValorMonster target
#Party party
AttackCommand
CastSpellCommand
TeleportCommand
-LovBoard board
-ValorHero hero
-int destRow
-int destCol
LovCommand
MoveCommand
-ValorHero hero
-int deltaRow
-int deltaCol
-LovBoard board
RecallCommand
-LovBoard board
-ValorHero hero
Attack
FightStrategy
CastSpell
LoVBattleProxy
-LoVBattle battle
-InventoryController inventoryController
~Scanner scanner
Battle
LoVBattle
-LivingEntity attacker
-LivingEntity target
-LovBoard board
-PartyController partyController
-LoVHeroController valorHeroController
-InventoryController inventoryController
BattleController
#Party party
#FightStrategy currentFightStrategy
#HeroController heroController
#InventoryController inventoryController
#InputHandler inputHandler
#Scanner scanner
MonstersAndHeroesBattleController
-InputHandler inputHandler
-List<Monster> monsters
-int roundCounter
-boolean battleActive
-HeroController heroController
HeroEventNotifier
PartyController
-List<HeroObserver> observers
HeroController
LoVHeroController
MHGameController
-Scanner scanner
-Board board
-GameView view
-MarketController marketController
-InventoryController inventoryController
-GuildController guildController
-HeroController heroController
GameEngine
GameController
#List<Hero> allHeroes
#List<Item> allItems
#List<Monster> allMonsters
#Party party
~Scanner scanner
~InputHandler inputHandler
#InventoryController inventoryController
LovGameController
-LovBoard board
-Scanner scanner
-LovGameState currentState
-int roundNumber
+boolean isRunning
+int numOfMonstersSpawned
-InputHandler inputHandler
-MarketController marketController
InventoryController
-Scanner scanner
+List<Spell> spells
+List<Weapon> weapons
GameView
GameLauncher
MonsterFactory
Main
Colors
BoardRenderer
MarketController
-Scanner scanner
-InventoryController inventoryController
-Market shop
GameDataParser
SoundPlayer
Party
GuildController
-Scanner scanner
-int swapsLeft
-HeroController heroController
GameDatabase
-GameDatabase instance
-List<Hero> allHeroes
-List<Monster> allMonsters
-List<Item> allItems
RandomTerrainGenerator
InputHandler
-Scanner scanner

4. Contributions

Ziyang:

- Map Generation and Layout:8×8 board, three lanes, fixed Nexus rows, central Inaccessible columns, balanced terrain mix.
- Terrain Buff System:Entry hooks, exit hooks, Dexterity on Bush, Agility on Cave, Strength on Koulou, obstacle-to-plain reset.
- Generation Integrity and Randomness Control:MapFactory wrapper, optional seed, rule checks, lane continuity validation, terrain quota audit.
- Terminal Visualization: ASCII renderer, tile abbreviations, H1–H3 overlays, M1–M3 overlays, shared-cell marker alignment.
- Monster Behavior: Adjacent attack check, southward advance, hero blocking respect, lane queue maintenance, future action hooks.
- Monster Spawning LifecyclePer-lane timer, N-round cadence, Nexus placement, level sync to party max, occupied-lane skip logic.

Chris:

- Combat System: Attacking, spell casting, and fight sequence orchestration via LoVBattle and LoVBattleProxy, hero respawning logic,
- Fight Mechanics: FightStrategy governs attack and spell-casting behavior; Attack and CastSpell implement concrete actions.
- Items & Equipment: Enhanced abstraction for spells and modifications to item/equipment handling.
- Shared Infrastructure: Centralized controllers, utilities, and pattern implementations for reuse across both games.
- Bug Fixes & Enhancements: Added supporting functionalities across multiple classes to facilitate fixes and ensure consistent gameplay behavior.
- Design Patterns Used: Strategy encapsulates combat behaviors, Proxy mediates battle access, Observer updates dependent systems on state changes

Serena:

- **Game Engine & Architecture:** Designed and implemented the `LovGameController` and the `GameLauncher`, serving as the backbone of the application.
- **State Machine Implementation:** Developed the `HeroTurnState`, `MonsterTurnState`, and `RoundEndState` classes to manage the strict turn-based flow and game phases.
- **Command System Logic:** Implemented the `LovCommand` interface and all concrete commands (`Move`, `Teleport`, `Recall`, `Attack`), including the complex logic for blocking movement behind enemies.

- **Integration Lead:** Merged the Board and Combat modules into the main engine. Implemented the "Status Dashboard" for real-time game stats.
- **Bonus Features:** Implemented the `SoundPlayer` for background music